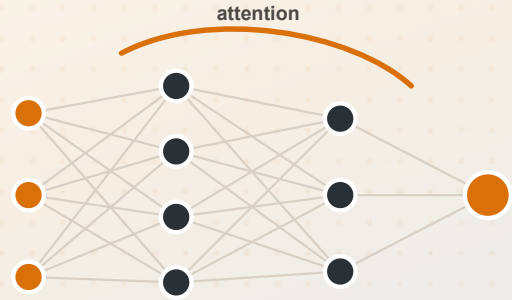


EPISODE 06

The Computational Brain of Machines

Neural networks, transformers, and attention



AI

NAMASTE AI - SEASON 1

Episode 06

The Computational Brain of Machines

This episode treats the neural network as the prediction engine inside an LLM and follows one input through embeddings, position, normalization, causal multi-head attention, residual pathways, feed-forward processing, logits, softmax, and repeated transformer layers.

02:31:26

WebVTT transcript only

In this episode

01 Next-token prediction as an iterative loop

02 GPT, transformers, and attention

03 Embeddings, position, and normalization

04 Causal multi-head self-attention

05 Residual pathways and feed-forward processing

06 Logits, softmax, and vocabulary-wide prediction

07 Q, K, V, projection, and stacked layers

08 Inference, model scale, code, and research reading

The neural network as a prediction engine

00:00 - 10:02

Machines can write poems, generate songs, videos, and images, solve exam questions, perform mathematics, and assist with research. What is the computation behind these capabilities? In the course's metaphor, the answer is the **neural network**-the machine's "computational brain."

The episode deliberately starts with the simplest useful description of an LLM:

1. receive some text;
2. predict the next token;
3. append that token to the text;
4. repeat.

The running prompt is:

The pizza is

Candidate continuations include plausible words such as `hot`, `ready`, `delicious`, `round`, and `cold`, alongside deliberately poor candidates such as `potato`, `bright`, `lazy`, and `fast`. The mock-up gives illustrative values including `hot = 0.68` and `ready = 0.75`.

The input side reuses the previous episode's representation pipeline:

`text → token IDs → token embeddings → neural network`

The output side contains one score for every possible next token. In the instructor's simplified framing, a higher value makes a token more likely to be selected.

Q: How does the model know that “ready” fits better than “potato”?

A: This episode assumes the neural network has already been trained. Its learned parameter values produce higher scores for continuations that fit the supplied context. How those values are learned is postponed to Episode 07.

Around `06:51–07:21`, the instructor briefly says a trained model makes the correct word's probability “lower,” then immediately and consistently explains that it should be high. The surrounding teaching flow supports “higher.”

Generation reuses the growing context

10:02 – 16:21

The example sentence develops step by step:

1 The pizza is produces ready .

2 The pizza is ready produces to , shown with the illustrative value 0.85 .

3 The pizza is ready to produces eat .

4 The expanded context may continue with with and then Coke .

The instructor repeatedly emphasizes that the model does not pass only the newest word back into the network. It processes the **whole available statement or context** again before predicting the next token.

Selection is also not presented as a rigid “always take the single largest number” rule. Several high-scoring tokens may be close or even tied, and another sampling or selection procedure may choose among them. That procedure is acknowledged but not explained here.

Punctuation such as a full stop may itself be represented as a token, and the lecture mentions termination logic that can end the response once enough information has been generated.

The categorical statement that “every character is a token” and “even a space is a token” conflicts with the course’s earlier variable-size-token explanation. It is preserved as a lecture statement, not promoted to a general definition.

What does GPT mean?

16:21 – 20:03

The acronym to remember is:

Generative

The model generates a continuation or output rather than merely retrieving an existing item.

Pre-trained

Before a learner chats with it, the model has already been trained on a large amount of data.

Transformer names the neural-network architecture explored through the rest of the episode. The instructor contrasts a search engine, which he describes as retrieving information, with an LLM, which generates text through repeated prediction.

The transformer and its central idea

20:03 – 30:00

The lecture traces the transformer to the 2017 paper **Attention Is All You Need**, described as a paper by eight researchers and connected in the instructor’s narration to Google’s AI research organization. Earlier sequence approaches named are **RNN** and **LSTM**, with LSTM expanded as *long short-term memory*.

The course definition is compact:

TRANSFORMER

A type of neural-network architecture designed to process sequences of information using attention.

The hierarchy is repeated several times:

LLM → transformer architecture → attention

The instructor calls the transformer the “heart” of the LLM and attention the “heart” of the transformer. He also uses a personal religious analogy involving Lord Hanuman, Lord Ram, and Lord Shiva to make the nesting memorable, while explicitly saying that it is an analogy rather than a religious lesson.

The instructor uses *GPT* as a broad category for Gemini, Grok, Claude, Llama, and every modern LLM because the acronym expands to “generative pre-trained transformer.” Preserve that as his taxonomy; the transcript does not supply evidence for turning it into a universal classification.

The exact relationship among “Google Brain,” “Google DeepMind,” and the paper team is also spoken with “I think.” The transcript does not support a firmer attribution.

Attention: which earlier information matters?

30:00 – 40:01

The first attention sentence is:

The cat sat on a mat because it was tired.

Humans infer that `it` refers to `cat`. The lecture says attention allows one token to consider its relationship with the other tokens in the sequence. To make that visible, the instructor supplies conceptual scores:

Relationship	Illustrative score
<code>it</code> ↔ <code>the</code>	0.1
<code>it</code> ↔ <code>cat</code>	0.9
<code>it</code> ↔ <code>sat</code>	0.2
<code>it</code> ↔ <code>mat</code>	0.4

The largest displayed relationship is between `it` and `cat`, supporting the intended reference. These are teaching values, not recorded output from a named model.

Self-attention means that tokens in one sequence examine relationships within that same sequence. “Self” refers to the sequence attending within itself; a human does not manually label every relationship.

The bank example provides a second view:

- “I went to the bank to deposit money.”
- “I sat on the bank of a river.”

`deposit` and `money` support the financial sense, while `sat` and `river` support the river-edge sense.

[Transcript unclear] A second cat/mat sentence involving the word “cozy” is badly garbled around `33:31–34:17`. It appears to test a different reference for `it`, but the exact sentence and conclusion cannot be recovered safely.

Token identity plus position

40:01 – 49:39

The architecture walk-through begins with tokenization, token IDs, and token embeddings. It then adds **positional embeddings** because word identity alone cannot preserve order.

Compare:

Sentence A

The dog bites a man.

Sentence B

The man bites a dog.

The token set is nearly the same, but the roles change. Token embeddings contribute identity; positional embeddings contribute location or order. The instructor says the two vectors are added element by element before entering the transformer.

Later, the interactive visualizer uses this toy input:

Token sequence	C	B	A	B	B	C
Token ID	2	1	0	1	1	2

Each ID is then represented by a **48-element vector** in the displayed nanoGPT example. The number **48** belongs to that visualized toy model, not to every transformer.

During this portion, the instructor pauses for a study habit: curiosity should produce questions. Rather than accepting each box in an architecture diagram, ask why it exists, what enters it, what it changes, and what would happen without it.

Layer normalization and numerical scale

49:39 – 54:34

Layer normalization is introduced as a way to keep values flowing through the network at a stable scale. Repeated additions, multiplications, attention, feed-forward work, and residual additions can make numbers grow.

The lecture uses intuitive progressions such as:

- $0.9 + 0.9 = 1.8$, followed by $1.8 + 5.2 = 7$;
- a vector that begins around $[0.2, 0.34, 0.5, 0.62]$ and later contains values such as $[22, 36, 14, 56]$.

Normalization is presented as a checkpoint that brings values back into a manageable range so deeper computation remains stable. In the visualizer, the instructor points to a mean, a standard deviation, and the learned quantities gamma and beta, described as weights and biases.

THE INSTRUCTOR LEAVES THIS OPEN

He asks whether reducing scale loses meaning, suggests that ratios may preserve it, uses $22/11 \rightarrow 2/1$ as intuition, and then explicitly says he is not sure. Do not treat the ratio example as the layer-normalization algorithm.

At `01:29:57`, the formula narration is partially preserved as “mean divide by variance,” followed by gamma and beta. **[Exact formula not available]** The VTT does not support reconstructing the normalization equation.

From self-attention to causal multi-head attention

54:34 – 01:00:04

The terminology grows one qualifier at a time:

1. **Self-attention:** tokens gather information from tokens in the same sequence.
2. **Causal or masked self-attention:** a position can use itself and the past, but not the future.
3. **Multi-head causal self-attention:** several attention operations run in parallel while obeying the same causal rule.

For next-token prediction, information must flow from the past toward the present. A token cannot use a future token that has not been generated yet. The lecture uses `My name is Akshay Saini` and later `I went to the bank to deposit money` to make this directional constraint concrete.

Why several heads? The instructor says different attention heads can inspect different relationships or aspects of the same sequence. The Bycroft visualization later displays three parallel heads: `head 1`, `head 2`, and `head 3`.

COMBINED LECTURE DEFINITION

Multi-head causal self-attention allows every token to gather information from previous tokens using multiple attention heads while preventing it from looking into the future.

Residual connections preserve the earlier representation

01:00:04 – 01:04:35

A bypass arrow around a layer is called a **residual connection** or **skip connection**. Instead of throwing away the incoming representation and replacing it completely, the network adds the original input back to the update produced by attention or the FFN.

The conceptual pattern is:

`original representation + useful update → updated representation`

The instructor starts with `[1, 2, 3]` and later names a result such as `[1.2, 1.5, 3.7]`. The exact intermediate update vector is not cleanly preserved. **[Transcript unclear]**

WHY THIS MATTERS IN THE LECTURE

A transformer layer influences what was already represented rather than discarding everything and starting again. Because addition can change numerical scale, normalization follows again.

The feed-forward network: per-token processing

01:04:35 – 01:10:00

After attention allows tokens to exchange contextual information, the **feed-forward network (FFN)** processes each token independently. The episode compresses the distinction into one memorable pair:

Attention

Lets tokens communicate.

FFN

Lets each token “think” and process what it learned.

Two jobs inside the block

The instructor's learner-friendly contrast

Attention

- tokens communicate
- relationships are gathered
- future positions stay masked
- multiple heads inspect aspects

FFN / MLP

- each token is processed
- the representation expands
- the representation contracts
- the token receives an update

The lecture's learner-friendly split: attention lets tokens communicate, while the FFN processes each token independently after that exchange.

For a token such as `bank`, attention gathers relationships from the permitted context. The FFN then updates or enriches that token's representation. The instructor pictures this as an expansion through hidden layers followed by a contraction to an enriched output.

The FFN is also called an **MLP**, or **multi-layer perceptron**. In the later visualizer, the instructor points out a linear transformation, the `GELU` activation function, and dot-product or cell-by-cell calculations.

“Thinking” and “expanding meaning” are explicitly layperson analogies. They are used to build intuition, not offered as research-precise definitions.

From logits to a probability distribution

01:10:00 – 01:18:29

After contextual processing, the model needs one score for every token in its vocabulary. The lecture introduces three connected terms:

- **linear layer or output projection:** produces raw numeric scores;
- **logits:** the raw scores before probability conversion;
- **softmax:** converts the scores to values between 0 and 1 whose total is 1.

The clean illustrative distribution is:

Token	Probability
cat	62%
dog	23%
car	10%
pizza	5%

The four displayed candidates sum to 100%, but the real conceptual output covers the **entire vocabulary**, not merely four choices. The instructor repeatedly uses an approximate size of 2 lakh tokens to communicate the scale; it is a lecture estimate, not a universal constant.

At 01:13:05, the instructor says the probabilities “sum up to 200,” then immediately corrects the explanation to 100% or 1.

The raw output scores are also called both embeddings and logits in a short passage. The lecture settles on **logits** for the pre-softmax output scores.

The complete transformer-block flow

01:18:29 – 01:32:18

The instructor now moves from a drawn overview to an interactive visualization credited to **Brendan Bycroft** at bbycroft.net. Only the root domain is spoken. **[Incomplete URL]** The exact visualization path is not present in the VTT.

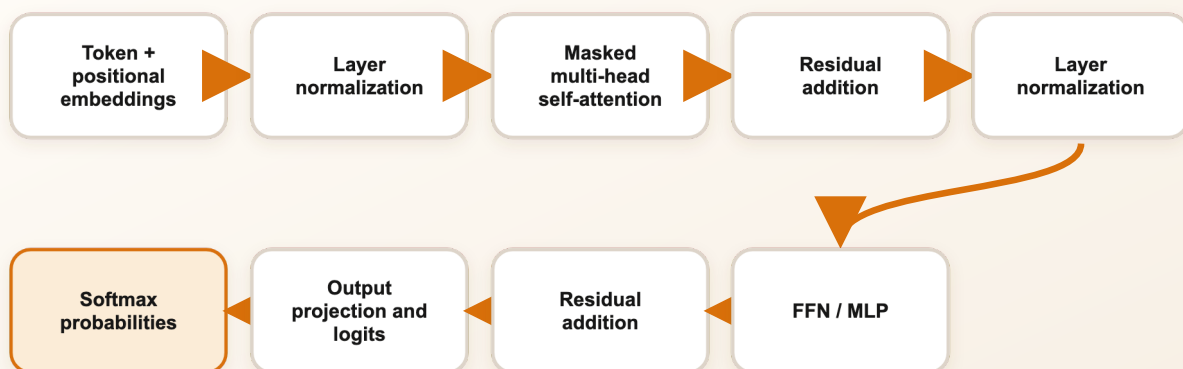
The conceptual flow assembled across the lecture is:

- 1 Tokenize the input and look up token embeddings.
- 2 Add positional embeddings.
- 3 Apply layer normalization.
- 4 Run multi-head masked or causal self-attention.

- 5 Add a residual pathway and normalize.
- 6 Run the feed-forward network or MLP.
- 7 Add another residual pathway and normalize.
- 8 Repeat the transformer block across multiple layers.
- 9 Use a linear projection to produce logits.
- 10 Apply softmax to obtain next-token probabilities.
- 11 Select one token, append it, and repeat the entire calculation.

One transformer-block walkthrough

The notes keep the lecture's order while separating attention, per-token processing, and output scoring



The lecture's conceptual transformer path separates token communication in attention, per-token processing in the FFN, and vocabulary-wide scoring at the output.

The interactive toy sequence **C B A B B C** makes the hidden numbers inspectable. It shows the input embedding, positional embedding, vector addition, and normalization before moving into attention. The instructor highlights a 48-element vector, mean and standard-deviation calculations, and gamma/beta values.

[Transcript unclear] Captions just before the Bycroft transition (**01:18:29-01:18:57**) and part of the normalization narration (**01:29:57-01:30:41**) are corrupted.

Query, key, and value

01:32:18 - 01:50:00

Inside each attention head, the visualizer introduces:

- **Q** = query;

- **K** = key;
- **V** = value;
- **O** = output.

Q, K, and V are described as learned numerical vectors or weights. Their operations include dot products, the creation of an attention matrix, softmax, and a value-side output.

To make key and value familiar, the instructor gives a JavaScript-object analogy:

```
// Code reconstructed from the unambiguous spoken example.
const user = {
  name: "Akshay"
};
```

Looking up `user.name` or `user["name"]` uses the key `name` to obtain the value `Akshay`. This is only a lookup intuition. It is not dictated transformer implementation code.

The bank narrative says a token “queries” its relationships with the other permitted tokens. Learned Q/K/V numbers participate in the comparisons that form attention weights; value vectors then contribute to the output.

ANALOGY BOUNDARY

The JavaScript object and the phrase “bank is the key and its embedding is the value” are rough teaching aids. They should not be treated as the formal mathematical definition of transformer Q, K, and V.

Why causal attention forms a triangle

01:50:00 – 01:56:19

Use the sequence:

```
I went to the bank to deposit money
```

Each position can attend only to itself and positions before it:

- **I** can use **I**;
- **went** can use **I** and **went**;
- the next **to** can use all earlier positions plus itself;
- later tokens continue the same pattern.

Future cells are masked or zeroed. The permitted cells therefore form a triangular attention matrix.

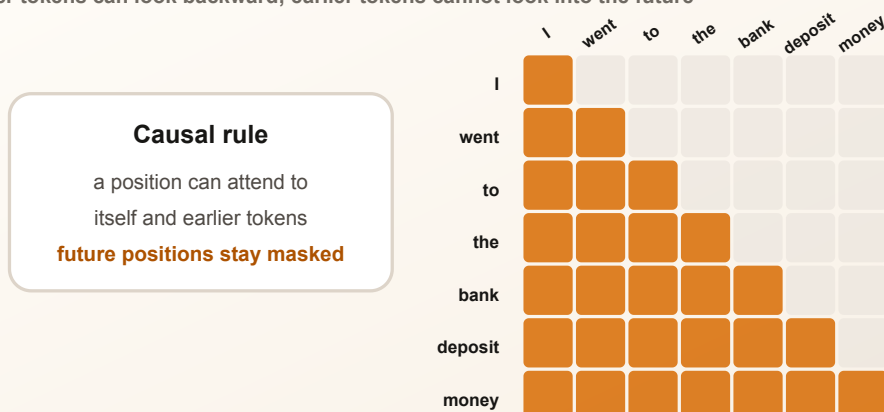
The instructor then raises a useful objection. If **bank** cannot look forward to the future token **money**, how can the completed sequence capture their relationship?

Q: How can “bank” and the later word “money” still become related?

A: While the model is processing the earlier position **bank**, it cannot attend to future **money**. When it later processes **money**, that position can attend backward to **bank**. The sequence can therefore represent the relationship without allowing information to flow from the future into the past.

Why causal attention forms a triangle

Later tokens can look backward; earlier tokens cannot look into the future



A position can use itself and earlier tokens, but not future ones. Later **money** can look back to **bank**; earlier **bank** cannot look forward to **money**.

The attention formula is visible on screen, and the instructor names **d_k** and **d_v** as query/key and value dimensions. **[Exact formula not available]** It is not fully dictated in the VTT.

Projection and repeated transformer layers

01:56:19 – 02:09:59

Each attention head produces an output. The outputs from the parallel heads are combined in an output or **projection** stage, then move through the residual pathway, normalization, and the FFN/MLP.

One complete group of these operations is treated as a transformer block or layer. An LLM stacks many such layers:

- an LLM contains repeated transformer layers;
- each layer contains multi-head attention;
- each head performs its own Q/K/V attention calculations.

The instructor compares the nesting to *Inception* or recursion. The output from one block becomes the input to the next. After the final layer, the output projection produces logits and softmax converts them to next-token probabilities.

This is not a one-time calculation. Every newly generated token expands the context and triggers the sequence again.

Inference and scale

02:10:00 – 02:18:12

The operation demonstrated throughout the episode is **inference**. The model's parameters are assumed to be trained; input enters, forward calculations run, and an output token is generated. Training-how the parameters became useful-is the subject of the next episode.

The visualizer compares nanoGPT, GPT-2 variants, and GPT-3. NanoGPT is said to have roughly 80–85 thousand parameters, but no repository version is specified. A passing comment also mentions GPT-5 and “6”; because the wording is time-sensitive and undefined, it is not used as an evergreen model comparison here.

The instructor's scale analogy moves from a human to Earth to the Milky Way. His substantive point is that the core kinds of operation remain similar while larger models repeat them over many more layers and parameter values. The original transformer paper's demonstrated setting is described here as translation, and the discussion connects model scale with GPUs and compute.

What the code demonstration does-and does not show

02:18:12 – 02:22:09

The instructor opens a nanoGPT repository and reports that `model.py` contains about 330 lines. The captions spell the repository author's name as “Andre Karpati.” **[Identifier unclear in transcript]** No exact repository URL, commit, or branch is supplied.

He points out sections for:

- layer normalization;
- causal self-attention;
- MLP or feed-forward work;
- matrix multiplication or dot-product operations;
- softmax or probability logic.

Some narration is itself uncertain-at one point the instructor says he is not sure whether a displayed section is the feed-forward network-and the captions are partly garbled. No exact function body should be inferred from this segment.

The conclusion is not that an LLM is “only a few lines.” The instructor separates the comparatively concise visible model code from the difficulty of the research, architecture, implementation, training data, and GPU compute behind it.

A method for reading research papers

02:22:09 – 02:29:42

The lecture next opens GPT-2 code and reports 174 lines in `src/models.py`, while acknowledging that related logic can be distributed through other files. Again, the VTT supplies no exact repository URL, version, or complete code.

The closing practice advice is a patient, question-driven reading process:

- 1 Read slowly, one information-dense sentence at a time.
- 2 When a term is unfamiliar, stop and investigate it before continuing.
- 3 After a first pass, ask an LLM to summarize the paper.
- 4 Compare that summary with your own understanding instead of accepting it blindly.
- 5 Ask follow-up questions and request references for concepts the paper does not explain.
- 6 Ask the LLM to quiz you; answer in writing or voice mode.
- 7 Repeat until the explanations and your understanding align.

The instructor reads the beginning of another paper about NLP tasks, supervised learning, and task-specific datasets. Its title and one dataset name are not reliably recoverable; captions render the identifier as “CEO QA.” **[Identifier unclear in transcript]**

Closing the inference loop

02:29:42 – 02:31:26

The chapter’s complete inference picture is:

```
input text → tokens → token and positional embeddings → repeated transformer layers  
→ normalization + masked multi-head attention + residuals + MLP → logits → softmax →  
selected next token
```

The course metaphor now has three nested levels: the neural network is the computational brain, the transformer is the central architecture being studied, and attention is its central mechanism. The instructor closes by repeating “Attention is all you need.”

Episode 07, **Sharpening the Brain**, moves from inference to training: how parameter values are adjusted so the correct next token receives a higher probability.

Chapter summary

An LLM generates by repeatedly predicting one next token from the entire available context. A trained neural network maps token representations to vocabulary-wide scores, and a selection step adds one token before the process runs again.

Inside the transformer architecture taught here, token embeddings contribute identity and positional embeddings contribute order. Layer normalization controls numerical scale. Masked self-attention allows a position to gather information from itself and the past, several heads perform that work in parallel, residual connections preserve earlier representations, and an FFN/MLP processes each token after communication. Repeated transformer blocks produce a final representation; a linear projection yields logits, and softmax turns them into next-token probabilities.

The interactive visualizer and code tours connect the architecture to numbers and implementation, but the instructor is careful to distinguish a conceptual overview from the full mathematics, research, data, and compute involved. This episode explains **inference**; training comes next.

Key takeaways

- Generation reprocesses the growing context and predicts one token at a time.
- GPT expands to Generative Pre-trained Transformer.
- Self-attention models relationships within one sequence; causal masking hides future positions.
- Multiple heads run in parallel and their outputs are combined through projection.
- Residual connections add useful updates without discarding the incoming representation.
- Attention lets tokens communicate; the FFN/MLP processes each token independently afterward.
- Logits are raw scores, and softmax converts them into a probability distribution.
- One transformer block is repeated across layers, and the whole forward process repeats for each generated token.

Revision questions

1. Walk through how “The pizza is” becomes one predicted next token.
2. Why is the entire growing sentence processed again instead of only the newest token?
3. What does each word in **Generative Pre-trained Transformer** mean?
4. Which paper and year does the lecture connect with the transformer architecture?
5. How do the illustrative attention scores resolve **it** in “The cat sat on a mat because it was tired”?
6. What does *self* mean in self-attention?
7. Why are positional embeddings added to token embeddings?
8. What does layer normalization try to keep stable, and what part of the lecture’s explanation remains uncertain?

9. Distinguish self-attention, causal or masked self-attention, and multi-head attention.
10. Why must a causal position not attend to a future token?
11. What does a residual or skip connection add back into a layer's update?
12. Complete the lecture's contrast: "Attention lets tokens communicate; FFN lets each token ..."
13. What does MLP stand for, and how is it used in this explanation?
14. What are logits, and what does softmax do to them?
15. Why must the output calculation cover the whole vocabulary rather than only the four displayed candidates?
16. What do Q, K, and V stand for?
17. Why does the causal attention matrix have a triangular shape?
18. How can the later word `money` capture a relationship with the earlier word `bank`?
19. What happens to the outputs of several attention heads?
20. Why is one transformer block not the whole LLM?
21. What is inference, and what assumption does this episode make about the model's parameters?
22. What did the Bycroft visualization add to the instructor's explanation?
23. Why do the reported `model.py` and `src/models.py` line counts not prove that building an LLM is easy?
24. Reproduce the instructor's research-paper reading method in order.